ELASTICSEARCH WORKSHOP

Category 6: Aggregations — Metrics

EXERCISES

Scenario

You are a data engineer for a fictional intelligence unit called UNIT-7.
Your unit operates surveillance drones. Each mission is logged with flight stats.
You will use metric aggregations to calculate statistics across all mission records.
All drone IDs, mission names, and locations are completely fictional.

Key Concept: Metric Aggregations

An aggregation calculates a statistic across a set of documents.

A metric aggregation calculates a single number: average, minimum, maximum, sum, or count.

Aggregations are separate from the query results. The query filters documents. The aggregation runs on those filtered documents.

Aggregation	What it returns	SQL equivalent
`avg`	The average value of a numeric field.	AVG(field)
`min`	The lowest value of a numeric field.	MIN(field)
`max`	The highest value of a numeric field.	MAX(field)
`sum`	The total of all values in a numeric field.	SUM(field)
`value_count`	How many documents have a value in a field.	COUNT(field)
`stats`	All of the above in one call.	AVG + MIN + MAX + SUM + COUNT
`extended_stats`	stats plus variance and std deviation.	No direct SQL equivalent

```
Structure of a query with aggregations:
 {
    'query': { ... filter documents ... },
    'aggs':  { 'my_agg_name': { 'avg': { 'field': 'field_name' } } }
 }

 'aggs' is the key for aggregations. You can also write 'aggregations'.
 'my_agg_name' is a label you choose. It appears in the response.
 The aggregation result is in: response['aggregations']['my_agg_name']

 Set 'size': 0 when you only want aggregation results and not document hits.
 This is faster because Elasticsearch skips returning documents.

SQL analogy:     SELECT AVG(field) FROM table WHERE ...
```

Dataset: Drone Mission Log

Index name: 'drone_missions' | 15 records

mission_id	drone_id	sector	mission_type	flight_hours	fuel_used_L	altitude_m	targets_identified	mission_date	successful
MSN-001	DRONE-A	NORTH	reconnaissance	4.5	320.0	3200	3	2024-01-05	true
MSN-002	DRONE-B	SOUTH	strike_support	2.1	180.0	2800	1	2024-01-09	true
MSN-003	DRONE-A	EAST	reconnaissance	6.2	440.0	3500	5	2024-01-14	true
MSN-004	DRONE-C	NORTH	reconnaissance	3.8	270.0	3100	2	2024-01-20	false
MSN-005	DRONE-B	WEST	strike_support	1.9	160.0	2600	0	2024-01-25	false
MSN-006	DRONE-A	SOUTH	patrol	5.0	350.0	3300	4	2024-02-02	true
MSN-007	DRONE-C	EAST	strike_support	2.5	210.0	2900	2	2024-02-07	true
MSN-008	DRONE-B	NORTH	patrol	4.1	290.0	3000	1	2024-02-12	true
MSN-009	DRONE-A	WEST	reconnaissance	7.3	510.0	3700	6	2024-02-18	true
MSN-010	DRONE-C	SOUTH	patrol	3.2	230.0	2800	3	2024-02-23	true
MSN-011	DRONE-B	EAST	reconnaissance	5.5	390.0	3400	4	2024-03-01	true
MSN-012	DRONE-A	NORTH	strike_support	2.8	240.0	3000	2	2024-03-06	true
MSN-013	DRONE-C	WEST	reconnaissance	4.9	340.0	3200	3	2024-03-11	false

MSN-014	DRONE-B	SOUTH	patrol	3.7	260.0	2900	2	2024-03-16	true
MSN-015	DRONE-A	EAST	patrol	4.2	300.0	3100	3	2024-03-21	true

Exercise 1 [Step-by-step]

Task: Create the 'drone_missions' index with the correct mapping. Bulk index all 15 records.

Step 1 — Create the index

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

if es.indices.exists(index='drone_missions'):
    es.indices.delete(index='drone_missions')

mapping = {
    'mappings': { 'properties': {
        'mission_id':          { 'type': 'keyword' },
        'drone_id':            { 'type': 'keyword' },
        'sector':              { 'type': 'keyword' },
        'mission_type':        { 'type': 'keyword' },
        'flight_hours':        { 'type': 'float'   },
        'fuel_used_L':         { 'type': 'float'   },
        'altitude_m':          { 'type': 'integer' },
        'targets_identified':  { 'type': 'integer' },
        'mission_date':        { 'type': 'date'    },
        'successful':          { 'type': 'boolean' }
    }}
}
es.indices.create(index='drone_missions', body=mapping)
print('Index created.')
```

Step 2 — Bulk index all 15 missions

```
from elasticsearch.helpers import bulk

missions = [

{'_index':'drone_missions','_id':'MSN-001','_source':{'mission_id':'MSN-001','drone
_id':'DRONE-A','sector':'NORTH','mission_type':'reconnaissance','flight_hours':4.5,
'fuel_used_L':320.0,'altitude_m':3200,'targets_identified':3,'mission_date':'2024-0
1-05','successful':True}},

{'_index':'drone_missions','_id':'MSN-002','_source':{'mission_id':'MSN-002','drone
_id':'DRONE-B','sector':'SOUTH','mission_type':'strike_support','flight_hours':2.1,
'fuel_used_L':180.0,'altitude_m':2800,'targets_identified':1,'mission_date':'2024-0
1-09','successful':True}},
```

{'_index':'drone_missions','_id':'MSN-003','_source':{'mission_id':'MSN-003','drone_id':'DRONE-A','sector':'EAST','mission_type':'reconnaissance','flight_hours':6.2,'fuel_used_L':440.0,'altitude_m':3500,'targets_identified':5,'mission_date':'2024-01-14','successful':True}},

{'_index':'drone_missions','_id':'MSN-004','_source':{'mission_id':'MSN-004','drone_id':'DRONE-C','sector':'NORTH','mission_type':'reconnaissance','flight_hours':3.8,'fuel_used_L':270.0,'altitude_m':3100,'targets_identified':2,'mission_date':'2024-01-20','successful':False}},

{'_index':'drone_missions','_id':'MSN-005','_source':{'mission_id':'MSN-005','drone_id':'DRONE-B','sector':'WEST','mission_type':'strike_support','flight_hours':1.9,'fuel_used_L':160.0,'altitude_m':2600,'targets_identified':0,'mission_date':'2024-01-25','successful':False}},

{'_index':'drone_missions','_id':'MSN-006','_source':{'mission_id':'MSN-006','drone_id':'DRONE-A','sector':'SOUTH','mission_type':'patrol','flight_hours':5.0,'fuel_used_L':350.0,'altitude_m':3300,'targets_identified':4,'mission_date':'2024-02-02','successful':True}},

{'_index':'drone_missions','_id':'MSN-007','_source':{'mission_id':'MSN-007','drone_id':'DRONE-C','sector':'EAST','mission_type':'strike_support','flight_hours':2.5,'fuel_used_L':210.0,'altitude_m':2900,'targets_identified':2,'mission_date':'2024-02-07','successful':True}},

{'_index':'drone_missions','_id':'MSN-008','_source':{'mission_id':'MSN-008','drone_id':'DRONE-B','sector':'NORTH','mission_type':'patrol','flight_hours':4.1,'fuel_used_L':290.0,'altitude_m':3000,'targets_identified':1,'mission_date':'2024-02-12','successful':True}},

{'_index':'drone_missions','_id':'MSN-009','_source':{'mission_id':'MSN-009','drone_id':'DRONE-A','sector':'WEST','mission_type':'reconnaissance','flight_hours':7.3,'fuel_used_L':510.0,'altitude_m':3700,'targets_identified':6,'mission_date':'2024-02-18','successful':True}},

{'_index':'drone_missions','_id':'MSN-010','_source':{'mission_id':'MSN-010','drone_id':'DRONE-C','sector':'SOUTH','mission_type':'patrol','flight_hours':3.2,'fuel_used_L':230.0,'altitude_m':2800,'targets_identified':3,'mission_date':'2024-02-23','successful':True}},

{'_index':'drone_missions','_id':'MSN-011','_source':{'mission_id':'MSN-011','drone_id':'DRONE-B','sector':'EAST','mission_type':'reconnaissance','flight_hours':5.5,'fuel_used_L':390.0,'altitude_m':3400,'targets_identified':4,'mission_date':'2024-03-01','successful':True}},

{'_index':'drone_missions','_id':'MSN-012','_source':{'mission_id':'MSN-012','drone_id':'DRONE-A','sector':'NORTH','mission_type':'strike_support','flight_hours':2.8,'fuel_used_L':240.0,'altitude_m':3000,'targets_identified':2,'mission_date':'2024-03-06','successful':True}},

{'_index':'drone_missions','_id':'MSN-013','_source':{'mission_id':'MSN-013','drone_id':'DRONE-C','sector':'WEST','mission_type':'reconnaissance','flight_hours':4.9,'fuel_used_L':340.0,'altitude_m':3200,'targets_identified':3,'mission_date':'2024-03-11','successful':False}},

{'_index':'drone_missions','_id':'MSN-014','_source':{'mission_id':'MSN-014','drone_id':'DRONE-B','sector':'SOUTH','mission_type':'patrol','flight_hours':3.7,'fuel_used_L':260.0,'altitude_m':2900,'targets_identified':2,'mission_date':'2024-03-16','successful':True}},

{'_index':'drone_missions','_id':'MSN-015','_source':{'mission_id':'MSN-015','drone_id':'DRONE-A','sector':'EAST','mission_type':'patrol','flight_hours':4.2,'fuel_use

```
d_L':300.0,'altitude_m':3100,'targets_identified':3,'mission_date':'2024-03-21','su
ccessful':True}},
]
success, errors = bulk(es, missions)
print('Indexed:', success, '| Errors:', errors)
```

Exercise 2 [Step-by-step] (required)

Task: Calculate the average flight_hours across all missions.

Use an avg aggregation. Set size to 0 so no documents are returned, only the aggregation result.

Step 1 — Write the aggregation query

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

# 'aggs' contains all aggregations.
# 'avg_flight_hours' is the name we choose for this aggregation.
# 'avg' is the aggregation type.
# 'field' is the numeric field to average.
# 'size': 0 means: do not return any documents, only the aggregation.
query = {
    'size': 0,
    'aggs': {
        'avg_flight_hours': {
            'avg': { 'field': 'flight_hours' }
        }
    }
}
```

Step 2 — Run and read the result

```
response = es.search(index='drone_missions', body=query)

# Aggregation results are in response['aggregations'].
# Use the name you gave the aggregation to access it.
# The value is in ['value'].
avg_hours = response['aggregations']['avg_flight_hours']['value']

print('Average flight hours:', round(avg_hours, 2))
# Expected output: Average flight hours: 4.11
# (4.5+2.1+6.2+3.8+1.9+5.0+2.5+4.1+7.3+3.2+5.5+2.8+4.9+3.7+4.2) / 15
```

Command reference:
'aggs': { 'your_name': { 'avg': { 'field': 'field_name' } } }

```
  response['aggregations']['your_name']['value']   The result.
  'size': 0   Skip returning documents. Faster when you only need aggregations.

SQL analogy:     SELECT AVG(flight_hours) FROM drone_missions
MongoDB analogy: db.drone_missions.aggregate([{ $group: { _id: null, avg: { $avg: '$flight_hours' } }
}])
```

Exercise 3

Task: Find the minimum and maximum altitude_m across all missions in one query.

Define two aggregations in the same query: one min and one max.
You can have as many named aggregations as you want in one 'aggs' block.

Print both results clearly.

Expected result: min altitude = 2600 m (MSN-005), max altitude = 3700 m (MSN-009).

```
Command reference:
  'aggs': {
      'lowest_altitude':  { 'min': { 'field': 'altitude_m' } },
      'highest_altitude': { 'max': { 'field': 'altitude_m' } }
  }

  Multiple aggregations run in the same request. Each has its own name.
  Access each by its name: response['aggregations']['lowest_altitude']['value']

SQL analogy:     SELECT MIN(altitude_m), MAX(altitude_m) FROM drone_missions
```

Exercise 4

Task: Calculate the total fuel used (fuel_used_L) across all missions.

Use a sum aggregation. Print the result in litres.

Expected result: 4260.0 litres total.

```
Command reference:
```

```
  'aggs': { 'total_fuel': { 'sum': { 'field': 'fuel_used_L' } } }
  response['aggregations']['total_fuel']['value']

SQL analogy:     SELECT SUM(fuel_used_L) FROM drone_missions
```

Exercise 5

Task: Count how many missions have a value in the targets_identified field.

Use a value_count aggregation. This counts documents where the field exists and has a value. It is different from the total document count.

Expected result: 15 (all missions have a value in targets_identified).

```
Command reference:
  'aggs': { 'mission_count': { 'value_count': { 'field': 'targets_identified' } } }
  response['aggregations']['mission_count']['value']

  value_count counts documents where the field is present and has a value.
  It does NOT count the total number of targets. Use sum for that.

SQL analogy:     SELECT COUNT(targets_identified) FROM drone_missions
```

Exercise 6 (required)

Task: Get all basic statistics (count, min, max, avg, sum) for flight_hours in one call.

Use the stats aggregation. It returns all five metrics at once.

Print each metric on its own line with a clear label.

Expected result: count=15, min=1.9, max=7.3, avg~4.11, sum=61.7.

```
Command reference:
  'aggs': { 'flight_stats': { 'stats': { 'field': 'flight_hours' } } }

  The result contains these keys:
    count   number of documents
```

```
  min     lowest value
  max     highest value
  avg     average value
  sum     total of all values

Access with: response['aggregations']['flight_stats']['avg'] etc.
```

Exercise 7 (required)

Task: Calculate the average targets_identified for successful missions only.

Add a filter to the query so only successful missions are included in the aggregation.
Use a bool filter for successful = true.

Expected result: average targets for successful missions only.
Successful missions: MSN-001(3), MSN-002(1), MSN-003(5), MSN-006(4), MSN-007(2), MSN-008(1), MSN-009(6), MSN-010(3), MSN-011(4), MSN-012(2), MSN-014(2), MSN-015(3). That is 12 missions.
Average = 36 / 12 = 3.0.

```
Command reference:
  Combine query and aggs in the same body:
  {
      'size': 0,
      'query': { 'bool': { 'filter': [ { 'term': { 'successful': True } } ] } },
      'aggs':  { 'avg_targets': { 'avg': { 'field': 'targets_identified' } } }
  }

  The query filters documents first. The aggregation runs only on those documents.
  This is like a SQL WHERE clause combined with an aggregate function.

SQL analogy:     SELECT AVG(targets_identified) FROM drone_missions WHERE successful = true
```

Exercise 8

Task: Calculate the total flight_hours for DRONE-A only.

Filter the query for drone_id = 'DRONE-A'. Then run a sum aggregation on flight_hours.

DRONE-A missions: MSN-001(4.5), MSN-003(6.2), MSN-006(5.0), MSN-009(7.3), MSN-012(2.8), MSN-015(4.2).

Expected result: sum = 30.0 hours.

```
Command reference:
  'query': { 'bool': { 'filter': [ { 'term': { 'drone_id': 'DRONE-A' } } ] } },
  'aggs':  { 'total_hours': { 'sum': { 'field': 'flight_hours' } } }

SQL analogy:     SELECT SUM(flight_hours) FROM drone_missions WHERE drone_id = 'DRONE-A'
```

Exercise 9

Task: Get extended statistics for fuel_used_L. Include standard deviation and variance.

Use the extended_stats aggregation. It includes everything in stats plus:

- std_deviation: how spread out the values are from the average.
- variance: the square of the standard deviation.
- std_deviation_bounds: the range within one standard deviation of the average.

Print the avg, std_deviation, and std_deviation_bounds (upper and lower).

```
Command reference:
  'aggs': { 'fuel_stats': { 'extended_stats': { 'field': 'fuel_used_L' } } }

  Extra keys in the result:
    std_deviation              How spread out the values are.
    variance                   std_deviation squared.
    std_deviation_bounds.upper  avg + std_deviation
    std_deviation_bounds.lower  avg - std_deviation

  Access with:
    result['std_deviation']
    result['std_deviation_bounds']['upper']
    result['std_deviation_bounds']['lower']
```

Exercise 10

Task: For reconnaissance missions only, calculate in one query: total missions (value_count), average flight_hours, total targets_identified (sum), and highest altitude (max).

Filter: mission_type = 'reconnaissance'

Define four aggregations in the same aggs block.

Reconnaissance missions: MSN-001, MSN-003, MSN-004, MSN-009, MSN-011, MSN-013. That is 6 missions.

Expected: count=6, avg flight_hours~5.37, total targets=23, max altitude=3700.

```
Command reference:
  'query': { 'bool': { 'filter': [ { 'term': { 'mission_type': 'reconnaissance' } } ] } },
  'aggs': {
      'recon_count':     { 'value_count': { 'field': 'mission_id'    } },
      'avg_hours':       { 'avg':         { 'field': 'flight_hours'  } },
      'total_targets':   { 'sum':         { 'field': 'targets_identified' } },
      'max_altitude':    { 'max':         { 'field': 'altitude_m'    } }
  }

  All four aggregations run in one request. Each returns its own result.

SQL analogy:
  SELECT COUNT(*), AVG(flight_hours), SUM(targets_identified), MAX(altitude_m)
  FROM drone_missions WHERE mission_type = 'reconnaissance'
```

End of Category 6 Exercises